

Integración SUMO-GEO + VaN3Twin

Visor web 3D (MapLibre/deck.gl) conectado en vivo a la simulación V2X de ns-3

Universidad de Cuenca — Redes Vehiculares y Heterogéneas (INGE-00104)

Prof. Pablo Barbecho — August 25, 2026

Contents

1	Objetivo y mecanismo	1
2	Arquitectura resultante	1
3	Mapa de la guía: qué se hace y dónde	1
4	Paso 1 — Parche en VaN3Twin: <code>setOrder(1)</code>	2
5	Paso 2 — Parche en SUMO-GEO: <code>setOrder(2)</code> y paso absoluto	3
6	Paso 3 — Versiones: <code>traci</code> del backend = SUMO del contenedor	4
7	Paso 4 — <code>docker-compose.van3twin.yml</code>	5
8	Paso 5 — Ejecución: el orden importa	6
9	Paso 6 — Verificación	7
10	Tarea del taller — 7 vehículos + vehículo de emergencia	7
11	Límites y notas de operación	8

1 Objetivo y mecanismo

Objetivo: que la plataforma **SUMO-GEO** (FastAPI + MapLibre GL JS + deck.gl, desplegada con Docker) visualice en 3D y en tiempo real *la misma simulación SUMO que controla VaN3Twin* (ns-3), en lugar de lanzar su propio SUMO.

Mecanismo: TraCI **multi-cliente**. SUMO admite varios clientes TraCI simultáneos si se arranca con `-num-clients N`; cada cliente declara su turno con `setOrder(k)` y la simulación avanza en *lockstep*: SUMO solo ejecuta un paso cuando *todos* los clientes lo han solicitado.

- **Cliente 1** → VaN3Twin (ns-3): controla la movilidad, inyecta los vehículos, ejecuta la pila ITS-G5 (CAM/DENM) y las métricas.
- **Cliente 2** → backend de SUMO-GEO en modo **remote**: *solo lee* (suscripciones de posición, velocidad, CO₂, semáforos) y alimenta el visor 3D del navegador.

SUMO-GEO ya incluye el modo `APP_SUMO_MODE=remote` para conectarse a un SUMO externo; la integración requiere únicamente **dos parches pequeños** (uno por lado) más el cableado Docker (red compartida y carpeta local montada).

2 Arquitectura resultante

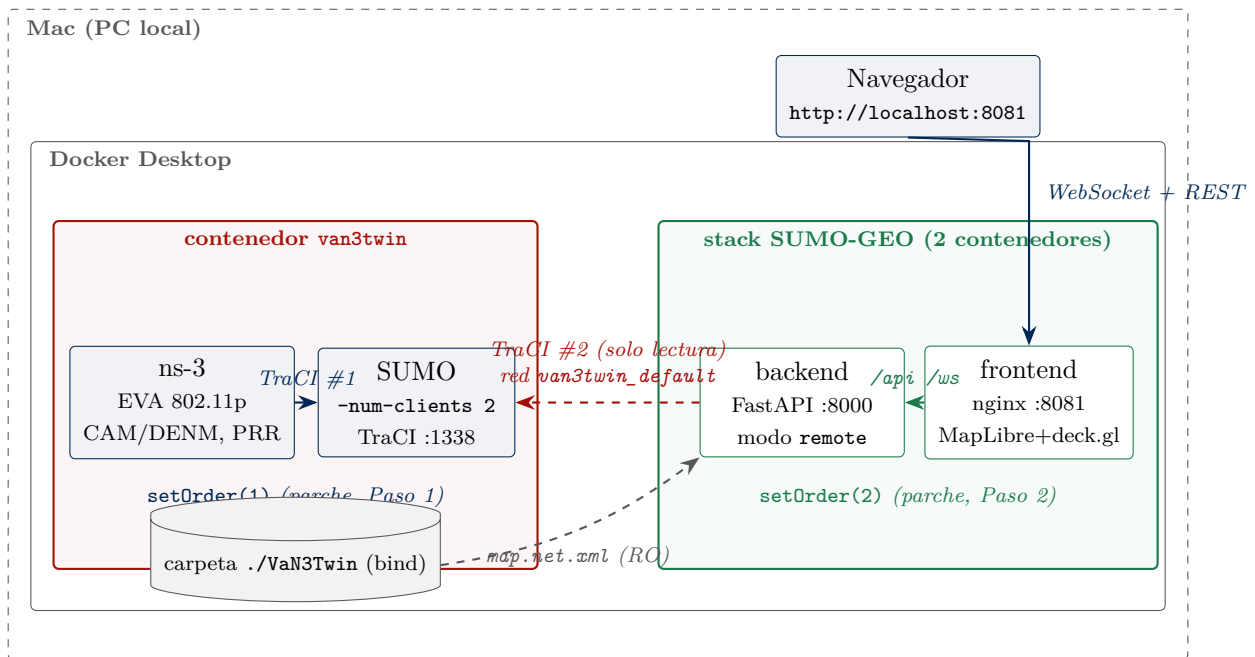


Figure 1: Arquitectura: dos proyectos Docker Compose independientes unidos por la red `van3twin_default` (TraCI) y la carpeta local `VaN3Twin` del Mac (bind mount, geometría del mapa).

Validado empíricamente

Antes de escribir esta guía se validó la mecánica en un entorno de pruebas (SUMO 1.27 + traci), con estos resultados:

1. **Lockstep sin bloqueo mutuo:** cliente 1 con pasos finos (0,1 s) + cliente 2 pidiendo objetivos gruesos (`simulationStep(t+1.0)`) $\Rightarrow$ el visor *no* frena a ns-3.
2. **El handshake del cliente 1 queda bloqueado** hasta que conecta el cliente 2: tras lanzar la simulación, ns-3 “espera” hasta abrir el visor.
3. **setOrder(1) es inocuo con un solo cliente:** el parche de VaN3Twin no altera el uso normal.
4. SUMO escucha TraCI en 0.0.0.0 $\Rightarrow$ es alcanzable desde otro contenedor por la red Docker **sin publicar el puerto** al Mac.

3 Mapa de la guía: qué se hace y dónde

Cada paso indica su ubicación con una caja de color: **rojo** = dentro del contenedor `van3twin`; **verde** = editar archivos del repo `SUMO_GEO` en el Mac; **azul** = terminal del Mac (PC local).

Nota: requisitos previos (Paso 0)

- Stack `van3twin-docker` construido y funcional (imagen `van3twin:jammy`).
- Repo `SUMO_GEO` clonado en el Mac: `git clone https://github.com/Pbarbecho/SUMO_GEO`.
- Puertos libres en el Mac: 8081 (frontend) y 8000 (backend). El puerto TraCI 1338 *no* se publica: viaja solo por la red interna de Docker.

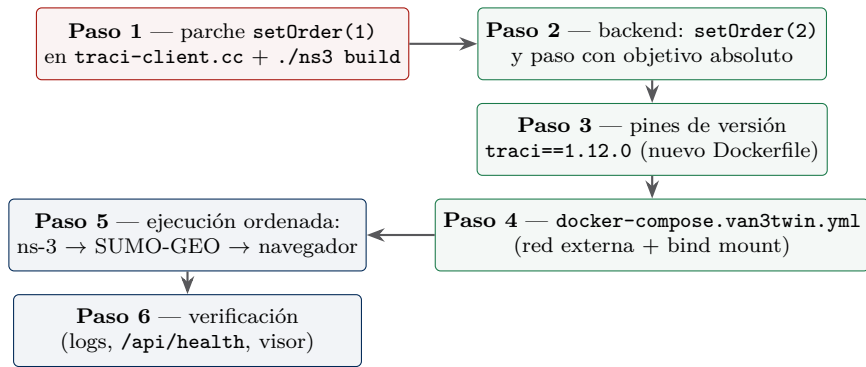


Figure 2: Secuencia de pasos y ubicación de cada uno (color).

4 Paso 1 — Parche en VaN3Twin: setOrder(1)

El cliente TraCI de VaN3Twin (`src/traci/model/traci-client.cc`) *nunca* llama a `setOrder()`, aunque su API lo implementa (`sumo-TraCIAPI.cc:87`). Sin esa llamada, SUMO multi-cliente no conoce el turno del cliente y la simulación se bloquea. El parche añade la llamada justo después de `TraCIAPI::connect()` (línea ≈ 282), **condicionada** a que se haya pedido `-num-clients`: el uso normal de VaN3Twin no cambia en absoluto.

❖ **DENTRO DEL CONTENEDOR `van3twin` — editar y recompilar (persiste en `./VaN3Twin` del Mac)**

```

# en el Mac: entrar al contenedor
cd ~/van3twin-docker && docker compose up -d
docker compose exec van3twin bash

# dentro del contenedor:
cd ~/VaN3Twin/ns-3-dev
sed -i '/this->TraCIAPI::connect("localhost", m_sumoPort);/a\
    if (m_sumoAddCmdOpt.find("--num-clients") != std::string::npos) { this->TraCIAPI::\
        setOrder(1); }' \
    src/traci/model/traci-client.cc

./ns3 build    # recompila solo el modulo traci y sus dependientes

```

El resultado, en contexto, queda así:

```

// connect to sumo via traci
try
{
    this->TraCIAPI::connect("localhost", m_sumoPort);
    if (m_sumoAddCmdOpt.find("--num-clients") != std::string::npos)
    { this->TraCIAPI::setOrder(1); }    // <-- NUEVO (parche)
}
catch (std::exception& e) { ... }

```

Nota: hacer el parche permanente (opcional)

El cambio anterior vive en la carpeta local `van3twin-docker/VaN3Twin` del Mac (bind mount), así que persiste siempre, incluso si se elimina el contenedor. Para hornearlo además en la imagen, añade el mismo `sed` como capa RUN en el Dockerfile de `van3twin-docker` (antes de la capa de `configure/build`) y reconstruye.

5 Paso 2 — Parche en SUMO-GEO: `setOrder(2)` y paso absoluto

Dos archivos del backend. Primero, un nuevo parámetro de configuración:

✂ EN EL MAC — editar archivos del repo SUMO_GEO — backend/app/config.py

Añadir tras `sumo_port`:

```
sumo_order: int = 0 # >0 => TraCI multi-cliente: llamar setOrder(N) al conectar
```

(Se controla con la variable de entorno `APP_SUMO_ORDER`; con el valor por defecto 0 el comportamiento actual no cambia.)

✂ EN EL MAC — editar archivos del repo SUMO_GEO — backend/app/sumo_bridge.py (2 cambios)

(a) En `start()`, rama `remote`:

```
if settings.sumo_mode == "remote" and not self._libsumo:
    client.init(host=settings.sumo_host, port=settings.sumo_port)
    if settings.sumo_order:
        client.setOrder(settings.sumo_order) # <-- NUEVO
    self.conn = client
```

(b) En `step()` — el cambio crítico:

```
def step(self) -> float:
    if settings.sumo_mode == "remote" and settings.sumo_order:
        # objetivo ABSOLUTO grueso: ns-3 marca el ritmo con sus pasos finos
        self.conn.simulationStep(
            self.conn.simulation.getTime() + settings.step_length)
    else:
        self.conn.simulationStep()
    # ... resto del metodo sin cambios ...
```

Por qué el objetivo absoluto es imprescindible

VaN3Twin lanza SUMO con `-step-length` igual a su intervalo de sincronización (pasos finos, p. ej. 0,01–0,1 s). En la barrera multi-cliente, si el visor pidiera “un paso” por frame (10 fps), ns-3 quedaría esclavo del visor: 10 pasos de 0,1 s por segundo real = simulación 10× más lenta. Con el objetivo absoluto `t+1.0`, el visor concede un margen amplio y ns-3 avanza a su propio ritmo (hecho validado #1).

*Barrera TraCI: SUMO avanza al ritmo del cliente más fino;
el objetivo grueso del visor nunca lo detiene.*

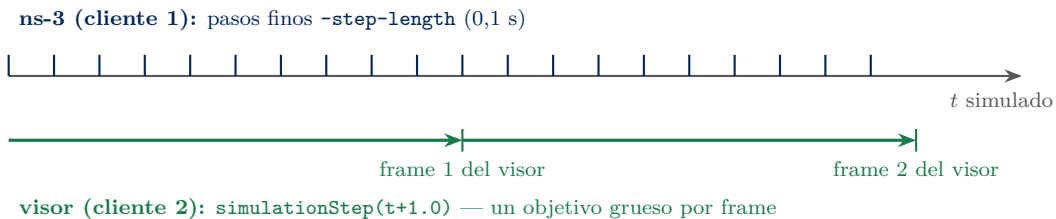


Figure 3: Lockstep multi-cliente: pasos finos (ns-3) frente a objetivos gruesos (visor).

6 Paso 3 — Versiones: traci del backend = SUMO del contenedor

El contenedor `van3twin` usa **SUMO 1.12.0** (apt de Ubuntu 22.04), pero el backend pina `traci==1.27.1`. El propio `requirements.txt` del repo lo advierte: “*Pin these to match the SUMO version you run*”. Además, en modo `remote` el backend *no necesita el binario* de SUMO (`eclipse-sumo`), solo las librerías puras de Python.

✂ EN EL MAC — editar archivos del repo SUMO_GEO —
backend/requirements-van3twin.txt (archivo nuevo)

```
fastapi==0.115.6
uvicorn[standard]==0.34.0
websockets==14.1
pydantic-settings>=2.4,<3
traci==1.12.0
sumolib==1.12.0
pyproj>=3.6
```

✂ EN EL MAC — editar archivos del repo SUMO_GEO —
backend/Dockerfile.van3twin (archivo nuevo)

Sin librerías X11/GL: no hay binario SUMO que las necesite.

```
FROM python:3.11-slim
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
WORKDIR /app
COPY requirements-van3twin.txt .
RUN pip install --no-cache-dir -r requirements-van3twin.txt
COPY app ./app
EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

7 Paso 4 — docker-compose.van3twin.yml

Archivo nuevo en la **raíz del repo SUMO_GEO**. Reutiliza la red *externa* `van3twin_default` (para TraCI; nombre derivado de `name: van3twin` en el compose de `van3twin`) y monta en solo lectura la carpeta local `~/van3twin-docker/VaN3Twin` (bind mount del stack `van3twin`, para leer el mapa). Verifica el nombre de la red antes:

■ EN EL MAC (PC local) — comprobar el nombre de la red

```
docker network ls | grep van3twin      # -> van3twin_default
```

✂ EN EL MAC — editar archivos del repo SUMO_GEO —
docker-compose.van3twin.yml (archivo nuevo, completo)

```
services:
  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile.van3twin
    environment:
      APP_SUMO_MODE: remote
```

```

APP_SUMO_HOST: van3twin          # container_name del stack van3twin
APP_SUMO_PORT: "1338"           # default de ns3::TraciClient::SumoPort
APP_SUMO_ORDER: "2"
# geometria: el MISMO net que usa el ejemplo EVA, leido del bind mount
APP_NET_FILE: /ns3/ns-3-dev/src/automotive/examples/sumo_files_v2v_map/map.net.xml
APP_POLY_FILE: /no_poly.xml      # inexistente a proposito (sin edificios)
APP_STEP_LENGTH: "1.0"          # objetivo de avance por frame del visor
APP_MAX_FPS: "10"

volumes:
  # ajustar la ruta si van3twin-docker esta en otro sitio
  - ~/van3twin-docker/VaN3Twin:/ns3:ro
ports:
  - "8000:8000"
networks: [default, van3twin]

frontend:
  image: nginx:1.27-alpine
  volumes:
    - ./frontend:/usr/share/nginx/html:ro
    - ./frontend/nginx.conf:/etc/nginx/conf.d/default.conf:ro
  ports:
    - "8081:80"
  depends_on: [backend]

networks:
  van3twin:
    external: true
    name: van3twin_default

```

Notas sobre el compose

- El **frontend no cambia**: nginx ya proxya /api y /ws al servicio backend.
- APP_POLY_FILE apunta a un archivo inexistente *a propósito*: si se deja vacío, main.py intentaría parsear /sumo/demo.sumocfg, que en este despliegue no está montado. (Mejora opcional: tolerar APP_POLY_FILE="" en main.py.)
- El backend pertenece a **dos redes**: la propia del stack (para nginx) y la externa van3twin_default (para TraCI).

8 Paso 5 — Ejecución: el orden importa

■ EN EL MAC (PC local) — 1) levantar van3twin

```
cd ~/van3twin-docker && docker compose up -d
```

⚡ DENTRO DEL CONTENEDOR van3twin — 2) lanzar la simulación EVA con 2 clientes TraCI

```

docker compose exec van3twin bash      # (desde el Mac)

# dentro del contenedor:
./ns3 run "v2v-emergencyVehicleAlert-80211p --sumo-gui=false --met-sup=true \
--ns3::TraciClient::SumoAdditionalCmdOptions='--num-clients 2'"
# -> ns-3 lanza SUMO y queda ESPERANDO al segundo cliente

```

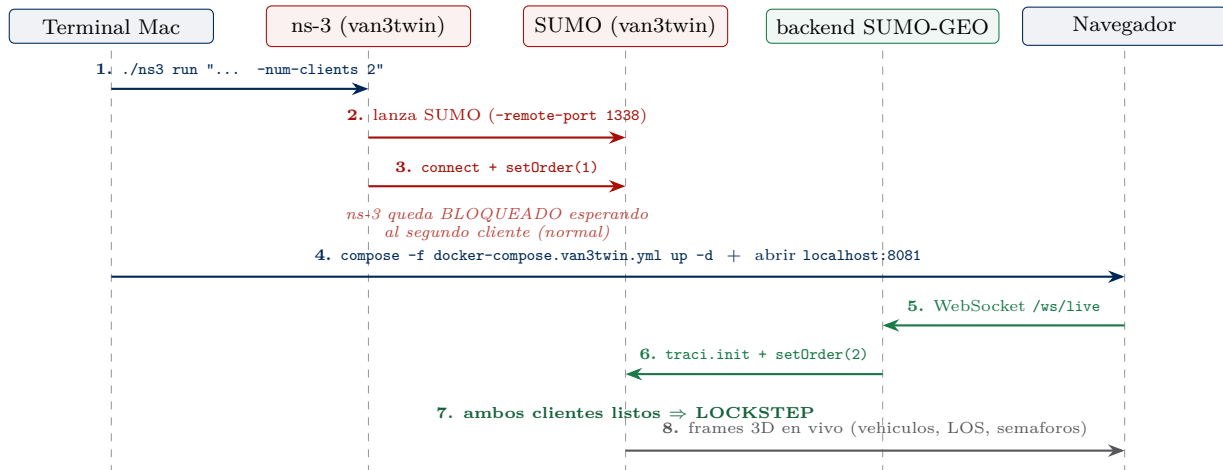


Figure 4: Secuencia de arranque. El paso 3 se bloquea hasta el paso 6 (hecho validado #2): por eso ns-3 debe lanzarse *antes* de abrir el visor.

La sintaxis `-ns3::TraciClient::SumoAdditionalCmdOptions=...` fija el atributo desde la línea de comandos *sin tocar el código del ejemplo*: el ejemplo EVA no asigna ese atributo y `traci-client.cc:240` concatena su valor al comando de SUMO. Puedes añadir tus flags habituales (`-csv-log`, `-penetrationRate`, ...).

■ EN EL MAC (PC local) — 3) levantar SUMO-GEO y abrir el visor

```
cd /ruta/a/SUMO_GEO
docker compose -f docker-compose.van3twin.yml up -d --build

open http://localhost:8081    # al abrirse el WebSocket arranca el lockstep
```

9 Paso 6 — Verificación

■ EN EL MAC (PC local) — comprobaciones

```
# backend sin errores de conexion TraCI ni de version:
docker compose -f docker-compose.van3twin.yml logs backend

# API viva:
curl http://localhost:8000/api/health
```

En el visor (`localhost:8081`) debe verse: los 7–8 vehículos del escenario EVA sobre el mapa real del ejemplo (zona de Turín) como cajas 3D orientadas, el color LOS por calle, el inspector de clic derecho (velocidad, *edge*, CO₂, espera...) y el panel de históricos. Al avanzar ns-3, los vehículos se mueven en el navegador *en el mismo instante simulado*.

Puerto (Mac)	Servicio	Stack
8081	Frontend SUMO-GEO (visor 3D)	SUMO-GEO
8000	Backend FastAPI (REST + WS)	SUMO-GEO
8080	vehicle-visualizer de VaN3Twin	van3twin
5901	VNC (GUI del contenedor)	van3twin
8090	GEO SUMO WEB (modo vivo, práctica)	van3twin
—	TraCI 1338: <i>no publicado</i> (solo red interna Docker)	ambos

10 Tarea del taller — 7 vehículos + vehículo de emergencia

Objetivo: preparar un escenario propio y observarlo en el visor 3D. Se parte del escenario de **7 vehículos** (`map_7.sumo.cfg`, que habilita `cars_7.rou.xml`) en `src/automotive/examples/sumo_files_v2v_map/`

Nota: editar con Visual Studio Code desde el Mac

Gracias al bind mount, los archivos de SUMO son archivos normales del Mac. Se recomienda abrirlos con VS Code — no hace falta editar dentro del contenedor ni recompilar nada: SUMO los lee al arrancar cada simulación.

```
code ~/van3twin-docker/VaN3Twin/ns-3-dev/src/automotive/examples/sumo_files_v2v_map
# (o VS Code -> File -> Open Folder... si el comando 'code' no esta en el PATH)
```

TAREA DEL TALLER

1. **Habilitar el escenario de 7 vehículos.** No hay que editar ningún `.sumocfg`: el ejemplo acepta `-sumo-config` y `map_7.sumo.cfg` ya apunta a `cars_7.rou.xml` (7 vehículos tipo `Car1`, 6,94 m/s).
2. **Añadir un vehículo de emergencia** en `cars_7.rou.xml`. Este fichero *no* define ningún tipo `emergency`: copiar el `vType Car0` del `cars.rou.xml` original y añadir la ambulancia como octavo vehículo:

```
<vType accel="4" decel="7.5" emergencyDecel="10" minGap="1.0"
      id="Car0" vClass="emergency" maxSpeed="20.83"/>
...
<vehicle id="veh8" type="Car0" depart="5" route="0">
</vehicle>
```

La clave es `vClass="emergency"`: la app la detecta con `vehicle.getVehicleClass()` (`emergencyVehicleAlert.cc:166`) y sus CAMs llevan `stationType = specialVehicle(10)`. Que salga *tarde* (`depart="5"`) y *rápida* (20,83 vs. 6,94 m/s) garantiza que alcance al pelotón.

3. **Misma ruta para todos.** Verificar/modificar que TODOS los `<vehicle>` llevan `route="0"` — la misma que la ambulancia. La reacción EVA exige distancia bajo umbral y rumbo similar: solo se dispara si comparten trayecto.
4. **Ejecutar con el visor 3D** (Pasos 5.2–5.3 de esta guía):

```
./ns3 run "v2v-emergencyVehicleAlert-80211p --sumo-gui=false --met-sup=true \
--sumo-config='src/automotive/examples/sumo_files_v2v_map/map_7.sumo.cfg' \
--ns3::TraciClient::SumoAdditionalCmdOptions='--num-clients 2'"
```

5. **Entregable:** captura del visor (`localhost:8081`) con la ambulancia alcanzando al pelotón + 3–5 líneas: ¿qué hacen los `Car1` al recibir sus CAMs (velocidad/carril con el inspector de clic derecho) y a qué distancia reaccionan?

11 Límites y notas de operación

- **Un visor por corrida.** `-num-clients` fija el número de clientes al arranque de SUMO. No usar el selector Bajo/Medio/Alto en este modo (reconectaría a un SUMO “gestionado” que aquí no existe); si se recarga la página a mitad de corrida, puede hacer falta relanzar el Paso 5.2.
- **Fin de la simulación.** VaN3Twin arranca SUMO con `-quit-on-end`: al terminar, SUMO se cierra y el visor pierde la conexión. Es lo esperado.
- **Qué muestra cada visor.** SUMO-GEO = *verdad terreno* de SUMO en 3D (todos los vehículos); GEO SUMO WEB 2D/3D (carpeta de la práctica) = *percepción por radio* (CAMs por receptor); `vehicle-visualizer (:8080)` = *verdad terreno* 2D nativa de VaN3Twin. Son complementarios.
- **Versiones.** Si se actualiza el SUMO del contenedor `van3twin`, actualizar los pines `traci/sumolib` del Paso 3 para que coincidan.
- **Rendimiento.** El ritmo lo marca `ns-3`; el visor añade una carga de lectura ligera (suscripciones TraCI, hecho validado #1). Con `APP_MAX_FPS=10` y `APP_STEP_LENGTH=1.0` el visor concede hasta 10 s simulados por segundo real: nunca es el cuello de botella.

Repos: https://github.com/Pbarbecho/SUMO_GEO · <https://github.com/DriveX-devs/VaN3Twin>. Guía complementaria: `GUIA_INTEGRACION_SUMO_GEO.md` y `manual VaN3Twin_Docker_macOS.pdf` en `van3twin-docker/`.